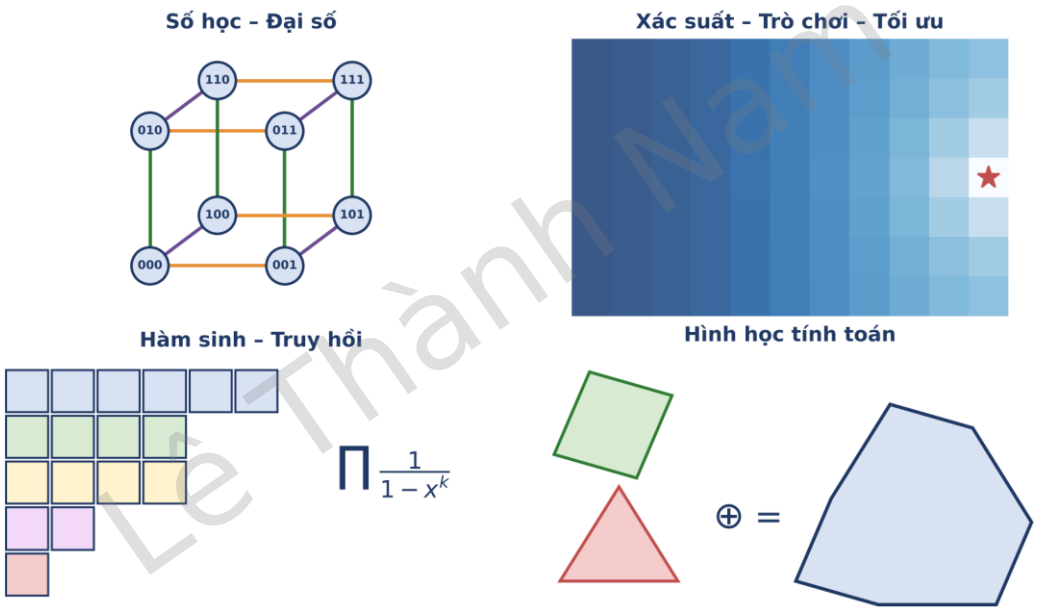
Tổng hợp và biên soạn: Lê Thành Nam

TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

TOÁN HỌC TRONG TIN HỌC

Chuyên đề nâng cao ôn thi học sinh giỏi Quốc gia (VOI) – Quyển 2

TẬP 3 · HƯỚNG DẪN GIẢI



Lê Thành Nam

Chương 1. SỐ HỌC VÀ ĐẠI SỐ NÂNG CAO
Chương 2. XÁC SUẤT – TRÒ CHƠI – TỐI ƯU
Chương 3. HÌNH HỌC TÍNH TOÁN NÂNG CAO

LỜI NÓI ĐẦU

Tập 3 trình bày lời giải cho 30 bài tập của Tập 2. Mỗi lời giải đi lại đúng con đường một học sinh giỏi suy nghĩ trong phòng thi: bắt đầu từ cách ngây thơ (thường đủ cho subtask đầu), đếm số phép tính để thấy bế tắc, thử một lời giải trung gian, rồi phá bế tắc bằng một quan sát hay một công cụ của Tập 1.

Mọi chương trình trong sách đều đã được kiểm tra hai lần: đối chiếu với lời giải vét cạn trên hàng trăm bộ dữ liệu nhỏ ngẫu nhiên, và chấm trên bộ test chính thức với giới hạn 1 giây, 256 MB. Các con số trong bảng điểm cuối mỗi bài là điểm thực tế mà từng chương trình nhận được trên bộ test đó, không phải ước lượng. Điểm của một subtask chỉ được tính khi chương trình đúng mọi test của subtask đó.

Lê Thành Nam

CẤU TRÚC MỘT LỜI GIẢI

Mục	Nội dung
1. Hiểu bài toán	Diễn đạt lại đề bằng ngôn ngữ toán học, nhận ra mô hình quen thuộc ẩn sau câu chuyện.
2. Cách ngây thơ	Ý tưởng, chương trình và độ phức tạp của cách làm đầu tiên ai cũng nghĩ ra.
3. Bế tắc	Vì sao cách hiện tại không vượt qua được subtask lớn; thường kèm một lời giải trung gian.
4. Phá bế tắc	Quan sát then chốt, công thức, chạy tay, lập luận vì sao đúng.
5. Cách tối ưu	Chương trình hoàn chỉnh và độ phức tạp.
6. Bẫy thường gặp	Những lỗi làm mất điểm hay gặp nhất.
7. Bảng điểm	Điểm thực tế của từng cách làm trên bộ test. Dòng tô xanh là lời giải mẫu.

Lời khuyên: Đừng đọc lời giải như đọc truyện. Sau mục 3, hãy gấp sách và tự tìm cách phá bế tắc trong ít nhất hai mươi phút. Sau mục 5, hãy tự viết lại chương trình mà không nhìn sách, rồi tự sinh test để đối chiếu với cách ngây thơ của chính em.

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**

CẤU TRÚC MỘT LỜI GIẢI **3**

CHƯƠNG 1. SỐ HỌC VÀ ĐẠI SỐ NÂNG CAO **5**

Bài 1.1. TỔNG SỐ NGUYÊN TỐ 6
Bài 1.2. TỔNG PHI 10
Bài 1.3. SỐ NỬA NGUYÊN TỐ 14
Bài 2.1. ĐẾM TẬP CON 18
Bài 2.2. XOR BẰNG KHÔNG 21
Bài 2.3. AND BẰNG KHÔNG 25
Bài 3.1. CHIA KẸO 29
Bài 3.2. PHÂN HOẠCH GIỚI HẠN 33
Bài 3.3. MẬT KHẨU CÂN BẰNG 37
Bài 4.1. TỔNG LŨY THỪA 41
Bài 4.2. DÃY LƯỢN SÓNG 45
Bài 4.3. ĐIỂM TRÊN CÂY 50
Bài 5.1. K CHẴNG 54
Bài 5.2. TRUY HỒI 57
Bài 5.3. LÁT GẠCH 61

CHƯƠNG 2. XÁC SUẤT – TRÒ CHƠI – TỐI ƯU **67**

Bài 6.1. MÊ CUNG 68
Bài 6.2. XUẤT HIỆN 71
Bài 6.3. RÔ BỐT 75
Bài 7.1. NIM MISÈRE 81
Bài 7.2. HỘP KẸO 85
Bài 7.3. MÈO VÀ CHUỘT 88
Bài 8.1. CHIA ĐOẠN 94
Bài 8.2. CẮT THANH 97
Bài 8.3. K ĐOẠN 100

CHƯƠNG 3. HÌNH HỌC TÍNH TOÁN NÂNG CAO **103**

Bài 9.1. ĐIỂM NGUYÊN 104
Bài 9.2. GẦN NHAU 107
Bài 9.3. CAMERA 110
Bài 10.1. VÒNG TRÒN 114
Bài 10.2. TAM GIÁC 118
Bài 10.3. VA CHẠM 121

Lê Thành Nam

CHƯƠNG 1. SỐ HỌC VÀ ĐẠI SỐ NÂNG CAO

Mỗi bài trong chương này được phân tích theo cùng một trình tự: hiểu bài toán, cách ngây thơ (kèm số điểm đạt được), chỗ bế tắc, các bước phá bế tắc, chương trình tối ưu, các bẫy thường gặp và bảng điểm đo thật của từng cách làm với giới hạn 1 giây mỗi test.

Lê Thành Nam

BÀI 1.1 TỔNG SỐ NGUYÊN TỐ

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
TONGNGTO.INP	TONGNGTO.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Bài 1

1. Hiểu bài toán

Cần $\Sigma_{p \le n,\ p \text{ nguyên tố}}\ p$ với n tới 10^{10}. Không thể sàng tới n (mảng 10^{10} phần tử). Đây đúng là Mở rộng "dạng tổng" của thuật toán Lucy ở Bài 1 (Tập 1): thay việc **đếm** số còn sống sót bằng việc **cộng** chúng.

2. Cách ngây thơ: Thử chia từng số

Cách ngây thơ: kiểm tra nguyên tố bằng thử chia, O(n√n)

```
#include <cstdio>
typedef long long ll;
const ll MOD = 1000000007;

int main() {
    if (fopen("TONGNGTO.INP", "r")) {
        freopen("TONGNGTO.INP", "r", stdin);
        freopen("TONGNGTO.OUT", "w", stdout);
    }
    ll n;
    scanf("%lld", &n);
    ll s = 0;
    for (ll x = 2; x <= n; x++) {
        bool prime = true;
        for (ll d = 2; d * d <= x; d++)
            if (x % d == 0) { prime = false; break; }
        if (prime) s = (s + x) % MOD;
    }
    printf("%lld\n", s);
    return 0;
}
```

Với $n = 10^6$ khoảng $7 \cdot 10^7$ phép chia, vừa kịp: đạt subtask 1. Với $n = 5 \cdot 10^7$ đã là $2{,}4 \cdot 10^{11}$.

3. Bế tắc

Lời giải trung gian tự nhiên là sàng Eratosthenes tới n với mảng char:

Lời giải trung gian: sàng Eratosthenes, O(n log log n)

```
#include <cstdio>
#include <vector>
using namespace std;
typedef long long ll;
const ll MOD = 1000000007;

int main() {
```

Lê Thành Nam

```
    if (fopen("TONGNGTO.INP", "r")) {
        freopen("TONGNGTO.INP", "r", stdin);
        freopen("TONGNGTO.OUT", "w", stdout);
    }
    ll n;
    scanf("%lld", &n);
    vector<char> comp(n + 1, 0);          // sang Eratosthenes
    ll s = 0;
    for (ll i = 2; i <= n; i++) {
        if (comp[i]) continue;
        s = (s + i) % MOD;
        for (ll j = i * i; j <= n; j += i) comp[j] = 1;
    }
    printf("%lld\n", s);
    return 0;
}
```

$n = 5 \cdot 10^7$: 50 MB và khoảng $1{,}2 \cdot 10^8$ thao tác, đạt subtask 2. Với $n = 10^{10}$ cần 10 GB bộ nhớ.

ĐIỂM BẾ TẮC

Phải trả lời mà không "chạm" vào từng số tới n. Thuật toán Lucy chỉ theo dõi khoảng $2\sqrt{n}$ giá trị $\lfloor n/i \rfloor$, mỗi giá trị lưu một **tổng** thay vì một số đếm.

4. Phá bế tắc: Lucy dạng tổng

Bước 1 Đại lượng theo dõi

Với $v \in V(n) = \{\lfloor n/i \rfloor\}$, gọi $g(v, p)$ là **tổng** các số trong $[2, v]$ là số nguyên tố hoặc có thừa số nguyên tố nhỏ nhất lớn hơn p. Ban đầu $g(v, 1) = 2 + 3 + \ldots + v = v(v+1)/2 - 1$.

Bước 2 Truy hồi

Khi chuyển sang số nguyên tố p, các số bị gạch có dạng $x = p \cdot y$ với y "sống sót" sau các số nguyên tố nhỏ hơn p và $y \geq p$. Tổng của chúng bằng p nhân tổng các y đó:

LUCY DẠNG TỔNG

$g(v, p) = g(v, p-1) - p \cdot (g(\lfloor v/p \rfloor, p-1) - g(p-1, p-1))$, chỉ áp dụng khi $p^2 \leq v$.

Kết quả: $g(n, \lfloor \sqrt{n} \rfloor)$ là tổng các số nguyên tố không vượt quá n.

So với dạng đếm, chỉ có thêm thừa số p (vì hàm $f(x) = x$ **nhân tính hoàn toàn**: $f(p \cdot y) = f(p) \cdot f(y)$). Mọi phép tính làm theo modulo $10^9 + 7$.

Bước 3 Chạy tay với n = 10

$V(10) = \{1, 2, 3, 5, 10\}$. Ban đầu g = 0, 2, 5, 14, 54 (tổng 2..v).

$p = 2$ (chỉ sửa $v \geq 4$): $g(5) = 14 - 2 \cdot (g(2) - g(1)) = 14 - 2 \cdot 2 = 10$ (bỏ 4); $g(10) = 54 - 2 \cdot (g(5)$ cũ $- 0) = 54 - 2 \cdot 14 = 26$ (bỏ 4, 6, 8, 10).

$p = 3$ (chỉ sửa $v \geq 9$): $g(10) = 26 - 3 \cdot (g(3) - g(2)) = 26 - 3 \cdot 3 = 17$ (bỏ 9). Kết quả $17 = 2 + 3 + 5 + 7$.

Lê Thành Nam

5. Cách tối ưu

TONGNGTO.CPP: Lucy dạng tổng theo modulo, O($n^{3/4}$/log n)

```cpp
#include <cmath>
#include <cstdio>
#include <vector>
using namespace std;
typedef long long ll;
const ll MOD = 1000000007, INV2 = (MOD + 1) / 2;

// tong 2 + 3 + ... + v theo modulo
ll S(ll v) {
    v %= MOD;
    return (v * ((v + 1) % MOD) % MOD * INV2 + MOD - 1) % MOD;
}

int main() {
    if (fopen("TONGNGTO.INP", "r")) {
        freopen("TONGNGTO.INP", "r", stdin);
        freopen("TONGNGTO.OUT", "w", stdout);
    }
    ll n;
    scanf("%lld", &n);
    ll r = sqrtl((long double)n);
    while (r * r > n) r--;
    while ((r + 1) * (r + 1) <= n) r++;
    // lo[v] = g(v), hi[i] = g(n / i); ban dau g(v) = 2 + 3 + ... + v
    vector<ll> lo(r + 1), hi(r + 1);
    for (ll v = 1; v <= r; v++) lo[v] = S(v);
    for (ll i = 1; i <= r; i++) hi[i] = S(n / i);
    for (ll p = 2; p <= r; p++) {
        if (lo[p] == lo[p - 1]) continue;          // p khong nguyen to
        ll sp = lo[p - 1], p2 = p * p;
        // loai cac hop so co thua so nho nhat la p
        ll lim = min(r, n / p2);
        for (ll i = 1; i <= lim; i++) {
            ll d = i * p;
            ll g = d <= r ? hi[d] : lo[n / d];
            hi[i] = (hi[i] - p * ((g - sp + MOD) % MOD)) % MOD;
            if (hi[i] < 0) hi[i] += MOD;
        }
        for (ll v = r; v >= p2; v--) {
            lo[v] = (lo[v] - p * ((lo[v / p] - sp + MOD) % MOD)) % MOD;
            if (lo[v] < 0) lo[v] += MOD;
        }
    }
    printf("%lld\n", n >= 1 ? hi[1] : 0LL);
    return 0;
}
```

Lê Thành Nam

Độ phức tạp: như dạng đếm, khoảng $4 \cdot 10^7$ phép cập nhật với $n = 10^{10}$, chạy dưới 0,2 giây. Bộ nhớ hai mảng long long độ dài 10^5.

6. Bẫy thường gặp

• **Tính v(v + 1)/2 trực tiếp** với $v = 10^{10}$: $v(v + 1) \approx 10^{20}$ tràn long long. Lấy v mod p trước rồi nhân với nghịch đảo của 2.

• **Nhận biết p nguyên tố bằng lo[p] ≠ lo[p − 1]** vẫn đúng khi lo lưu tổng theo modulo, vì hiệu bằng p, khác 0 theo modulo $10^9 + 7$.

- **Quên thừa số p** trong công thức cập nhật (chép nguyên dạng đếm): sai ngay từ n = 4.
- **Kết quả âm** sau phép trừ theo modulo: cộng thêm MOD.

7. Bảng điểm theo thuật toán

Thuật toán	Subtask 1 (20)	Subtask 2 (30)	Subtask 3 (50)	Tổng
Thử chia từng số, O(n√n)	20	0	0	**20**
Sàng Eratosthenes, O(n log log n)	20	30	0	**50**
Thuật toán Lucy trên các giá trị [n/i], O($n^{3/4}$)	20	30	50	**100**

Lê Thành Nam

BÀI 1.2 TỔNG PHI

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
TONGPHI.INP	TONGPHI.OUT	1 giây	256 MB	★★ TRUNG BÌNH

Liên hệ: Bài 1

1. Hiểu bài toán

Cần $\Phi(n) = \Sigma_{i \le n}\, \varphi(i)$ với n tới 10^{10} theo modulo $10^9 + 7$. Hàm φ nhân tính nhưng không nhân tính hoàn toàn, nên Lucy không áp dụng trực tiếp được. Công cụ đúng là sàng Đỗ Giáo, dựa trên đẳng thức $\Sigma_{d \mid k}\, \varphi(d) = k$.

2. Cách ngây thơ: Tính φ(i) bằng thử chia

Cách ngây thơ: phân tích từng i, O(n√n)

```cpp
#include <cstdio>
typedef long long ll;
const ll MOD = 1000000007;

ll phi(ll x) {                          // phan tich bang thu chia
    ll r = x;
    for (ll d = 2; d * d <= x; d++)
        if (x % d == 0) {
            while (x % d == 0) x /= d;
            r -= r / d;
        }
    if (x > 1) r -= r / x;
    return r;
}

int main() {
    if (fopen("TONGPHI.INP", "r")) {
        freopen("TONGPHI.INP", "r", stdin);
        freopen("TONGPHI.OUT", "w", stdout);
    }
    ll n;
    scanf("%lld", &n);
    ll s = 0;
    for (ll i = 1; i <= n; i++) s = (s + phi(i)) % MOD;
    printf("%lld\n", s);
    return 0;
}
```

Đạt subtask 1 ($n \le 10^4$).

3. Bế tắc

Sàng tuyến tính (Quyển 1) tính mọi φ(i) với $i \le n$ trong O(n):

Lê Thành Nam

Lời giải trung gian: sàng tuyến tính, O(n)

```cpp
#include <cstdio>
#include <vector>
using namespace std;
typedef long long ll;
const ll MOD = 1000000007;

int main() {
    if (fopen("TONGPHI.INP", "r")) {
        freopen("TONGPHI.INP", "r", stdin);
        freopen("TONGPHI.OUT", "w", stdout);
    }
    ll n;
    scanf("%lld", &n);
    vector<int> phi(n + 1), pr;          // sang tuyen tinh
    vector<char> comp(n + 1, 0);
    ll s = n >= 1 ? 1 : 0;
    for (ll i = 2; i <= n; i++) {
        if (!comp[i]) { pr.push_back(i); phi[i] = i - 1; }
        for (int p : pr) {
            if (i * p > n) break;
            comp[i * p] = 1;
            if (i % p == 0) { phi[i * p] = phi[i] * p; break; }
            phi[i * p] = phi[i] * (p - 1);
        }
        s = (s + phi[i]) % MOD;
    }
    printf("%lld\n", s);
    return 0;
}
```

Với $n = 10^7$: mảng int 40 MB, chạy khoảng 0,1 giây: đạt subtask 2. Với $n = 10^{10}$ cần 40 GB.

> **ĐIỂM BẾ TẮC**
>
> Phải tính tổng tiền tố của φ tại n mà chỉ sàng tới một ngưỡng $K \ll n$.

4. Phá bế tắc: Sàng Đỗ Giáo

Bước 1 Đẳng thức đệ quy

Cộng đẳng thức $\Sigma_{d \mid k} \varphi(d) = k$ với k = 1..x, rồi nhóm theo e = k/d: mỗi cặp (d, e) với $d \cdot e \le x$ đóng góp φ(d). Với mỗi e cố định, d chạy tới ⌊x/e⌋, nên:

> **SÀNG ĐỖ GIÁO CHO φ**
>
> $\Sigma_{e=1..x} \Phi(\lfloor x/e \rfloor) = x(x+1)/2$, suy ra $\Phi(x) = x(x+1)/2 - \Sigma_{e=2..x} \Phi(\lfloor x/e \rfloor)$.

Bước 2 Chia khối và ghi nhớ

Tổng bên phải có $O(\sqrt{x})$ giá trị ⌊x/e⌋ khác nhau (chia khối). Mọi đối số trong quá trình đệ quy đều có dạng ⌊n/i⌋, nên ghi nhớ bằng mảng theo chỉ số i = n/x. Sàng sẵn Φ tới $K \approx n^{2/3} \approx 4{,}6 \cdot 10^6$.

Lê Thành Nam

Bước 3 **Chạy tay với x = 10**

Các giá trị $\lfloor 10/e \rfloor$ với e = 2..10 là 5, 3, 2, 2, 1, 1, 1, 1, 1. Với $\Phi(5) = 10$, $\Phi(3) = 4$, $\Phi(2) = 2$, $\Phi(1) = 1$:

$\Phi(10) = 55 - (10 + 4 + 2 \cdot 2 + 5 \cdot 1) = 55 - 23 = 32$. Khớp với ví dụ.

5. Cách tối ưu

TONGPHI.CPP: Sàng Đỗ Giáo, $O(n^{2/3})$

```cpp
#include <cmath>
#include <cstdio>
#include <vector>
using namespace std;
typedef long long ll;
const ll MOD = 1000000007, INV2 = (MOD + 1) / 2;

ll n, K;                      // K: nguong sang truc tiep, khoang n^(2/3)
vector<ll> pre;               // pre[i] = phi(1) + ... + phi(i) mod p
vector<ll> big;               // big[n / x] = Phi(x) voi x > K
vector<char> done;

ll T(ll x) { x %= MOD; return x * ((x + 1) % MOD) % MOD * INV2 % MOD; }

ll Phi(ll x) {
    if (x <= K) return pre[x];
    ll id = n / x;
    if (done[id]) return big[id];
    // Phi(x) = x(x+1)/2 - sum_{d=2..x} Phi(x / d), chia khoi theo x / d
    ll res = T(x);
    for (ll d = 2; d <= x;) {
        ll q = x / d, e = x / q;
        res = (res - (e - d + 1) % MOD * Phi(q)) % MOD;
        d = e + 1;
    }
    res = (res % MOD + MOD) % MOD;
    done[id] = 1;
    return big[id] = res;
}

int main() {
    if (fopen("TONGPHI.INP", "r")) {
        freopen("TONGPHI.INP", "r", stdin);
        freopen("TONGPHI.OUT", "w", stdout);
    }
    scanf("%lld", &n);
    K = max(10LL, (ll)pow((double)n, 2.0 / 3));
    if (K > n) K = n;
    vector<int> phi(K + 1), pr;
    vector<char> comp(K + 1, 0);
    pre.assign(K + 1, 0);
    if (K >= 1) phi[1] = 1;
    for (ll i = 2; i <= K; i++) {
        if (!comp[i]) { pr.push_back(i); phi[i] = i - 1; }
        for (int p : pr) {
            if (i * p > K) break;
            comp[i * p] = 1;
            if (i % p == 0) { phi[i * p] = phi[i] * p; break; }
            phi[i * p] = phi[i] * (p - 1);
        }
    }
    for (ll i = 1; i <= K; i++) pre[i] = (pre[i - 1] + phi[i]) % MOD;
    big.assign(n / (K + 1) + 2, 0);
```

Lê Thành Nam

```
    done.assign(n / (K + 1) + 2, 0);
    printf("%lld\n", Phi(n));
    return 0;
}
```

Độ phức tạp: sàng tới $K = n^{2/3}$ tốn $O(K)$; phần đệ quy tốn $\Sigma \sqrt{(n/i)}$ trên $i \le n/K$, cũng $O(n^{2/3})$. Với $n = 10^{10}$ khoảng 10^7 phép tính.

6. Bẫy thường gặp

- **Tràn số ở x(x + 1)/2** và ở $(e_2 - e_1 + 1) \cdot \Phi(q)$: lấy modulo trước khi nhân.
- **Ghi nhớ bằng unordered_map:** đúng nhưng chậm hơn nhiều; mảng theo chỉ số n/x gọn và nhanh.
- **Chọn K = √n:** đệ quy sâu và chậm. K quanh $n^{2/3}$ là tối ưu.

7. Bảng điểm theo thuật toán

Thuật toán	Subtask 1 (20)	Subtask 2 (30)	Subtask 3 (50)	Tổng
Tính φ(i) bằng thử chia, O(n√n)	20	0	0	**20**
Sàng tuyến tính, O(n)	20	30	0	**50**
Sàng Đỗ Giáo, $O(n^{2/3})$	20	30	50	**100**

Lê Thành Nam

BÀI 1.3 SỐ NỬA NGUYÊN TỐ

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
NUANGTO.INP	NUANGTO.OUT	1 giây	256 MB	★★★ NÂNG CAO

Liên hệ: Bài 1

1. Hiểu bài toán

Đếm số cặp (p, q) nguyên tố với $p \le q$ và $p \cdot q \le n$. Với mỗi p, q chạy trên các số nguyên tố trong $[p, \lfloor n/p \rfloor]$; vì $p \le q$ nên $p \le \sqrt{n}$. Đáp số:

> **CÔNG THỨC**
>
> Đáp số = $\Sigma_{p \le \sqrt{n},\ p \text{ nguyên tố}} \left(\pi(\lfloor n/p \rfloor) - \pi(p) + 1 \right)$.

Cần π tại các giá trị $\lfloor n/p \rfloor$ (lớn) và tại p (nhỏ): đúng các giá trị mà thuật toán Lucy tính ra **cùng một lúc**.

2. Cách ngây thơ: Thử chia từng số

Cách ngây thơ: đếm thừa số nguyên tố của từng số, O(n√n)

```cpp
#include <cstdio>
typedef long long ll;

int main() {
    if (fopen("NUANGTO.INP", "r")) {
        freopen("NUANGTO.INP", "r", stdin);
        freopen("NUANGTO.OUT", "w", stdout);
    }
    ll n;
    scanf("%lld", &n);
    ll cnt = 0;
    for (ll x = 2; x <= n; x++) {
        ll y = x;
        int k = 0;                          // so thua so nguyen to (ke ca lap)
        for (ll d = 2; d * d <= y && k < 3; d++)
            while (y % d == 0) { y /= d; k++; }
        if (y > 1) k++;
        if (k == 2) cnt++;
    }
    printf("%lld\n", cnt);
    return 0;
}
```

Đạt subtask 1.

Lê Thanh Nam

3. Bế tắc

Sàng tuyến tính đếm số thừa số nguyên tố $\Omega(x)$ (kể cả bội) cho mọi $x \le n$, rồi đếm các x có $\Omega(x) = 2$:

Lời giải trung gian: sàng tuyến tính tính Ω(x), O(n)

```cpp
#include <cstdio>
#include <vector>
using namespace std;
typedef long long ll;

int main() {
    if (fopen("NUANGTO.INP", "r")) {
        freopen("NUANGTO.INP", "r", stdin);
        freopen("NUANGTO.OUT", "w", stdout);
    }
    ll n;
    scanf("%lld", &n);
    // om[x] = so thua so nguyen to cua x (ke ca lap), sang tuyen tinh
    vector<char> om(n + 1, 0);
    vector<int> pr;
    ll cnt = 0;
    for (ll i = 2; i <= n; i++) {
        if (om[i] == 0) { om[i] = 1; pr.push_back(i); }
        if (om[i] == 2) cnt++;
        for (int p : pr) {
            if (i * p > n) break;
            om[i * p] = om[i] + 1;
            if (i % p == 0) break;
        }
    }
    printf("%lld\n", cnt);
    return 0;
}
```

Với $n = 2 \cdot 10^7$ đạt subtask 2. Với $n = 10^{11}$ không thể sàng.

> **ĐIỂM BẾ TẮC**
>
> Phải đếm mà không liệt kê. Công thức ở mục 1 chỉ cần π tại $O(\sqrt{n})$ điểm.

4. Phá bế tắc: Lucy một lần, dùng hai lần

Chạy Lucy dạng đếm: sau khi chạy, $lo[v] = \pi(v)$ với $v \le r = \lfloor\sqrt{n}\rfloor$ và $hi[i] = \pi(\lfloor n/i \rfloor)$. Với $p \le r$ thì $\lfloor n/p \rfloor$ chính là $hi[p]$, còn $\pi(p)$ là $lo[p]$. Không cần thêm bất kì phép tính π nào.

Chạy tay $n = 30$ ($r = 5$): $p = 2$: $\pi(15) - \pi(2) + 1 = 6 - 1 + 1 = 6$ ($q = 2, 3, 5, 7, 11, 13$). $p = 3$: $\pi(10) - \pi(3) + 1 = 4 - 2 + 1 = 3$ ($q = 3, 5, 7$). $p = 5$: $\pi(6) - \pi(5) + 1 = 3 - 3 + 1 = 1$ ($q = 5$). Tổng 10.

Lê Thành Nam

5. Cách tối ưu

NUANGTO.CPP: Lucy + cộng theo p ≤ √n, O($n^{3/4}$/log n)

```cpp
#include <cmath>
#include <cstdio>
#include <vector>
using namespace std;
typedef long long ll;

int main() {
    if (fopen("NUANGTO.INP", "r")) {
        freopen("NUANGTO.INP", "r", stdin);
        freopen("NUANGTO.OUT", "w", stdout);
    }
    ll n;
    scanf("%lld", &n);
    ll r = sqrtl((long double)n);
    while (r * r > n) r--;
    while ((r + 1) * (r + 1) <= n) r++;
    // Lucy: lo[v] = pi(v), hi[i] = pi(n / i)
    vector<ll> lo(r + 2), hi(r + 2);
    for (ll v = 1; v <= r; v++) lo[v] = v - 1;
    for (ll i = 1; i <= r; i++) hi[i] = n / i - 1;
    for (ll p = 2; p <= r; p++) {
        if (lo[p] == lo[p - 1]) continue;
        ll sp = lo[p - 1], p2 = p * p;
        ll lim = min(r, n / p2);
        for (ll i = 1; i <= lim; i++) {
            ll d = i * p;
            hi[i] -= (d <= r ? hi[d] : lo[n / d]) - sp;
        }
        for (ll v = r; v >= p2; v--) lo[v] -= lo[v / p] - sp;
    }
    // x = p * q voi p <= q: dem q nguyen to trong [p, n / p]
    ll cnt = 0;
    for (ll p = 2; p <= r; p++)
        if (lo[p] != lo[p - 1]) cnt += hi[p] - lo[p] + 1;
    printf("%lld\n", cnt);
    return 0;
}
```

Lê Thành Nam

Độ phức tạp: Lucy với n = 10^{11} khoảng 0,3 giây; phần cộng chỉ O(√n).

6. Bẫy thường gặp

- **Đếm cả p > q** (đếm mỗi số hai lần) hoặc quên trường hợp p = q (số chính phương của số nguyên tố như 25).
- **Dùng sqrt trên double** cho r: với n gần 10^{11} có thể lệch một đơn vị; chỉnh lại bằng hai vòng while.
- **Lấy π(⌊n/p⌋) từ lo[] khi ⌊n/p⌋ > r:** chỉ số vượt mảng. Với p ≤ r, luôn đọc hi[p].